



Elastic Observability

Principal Support Engineer

Jigar Shah

Observability Concepts



What is Observability

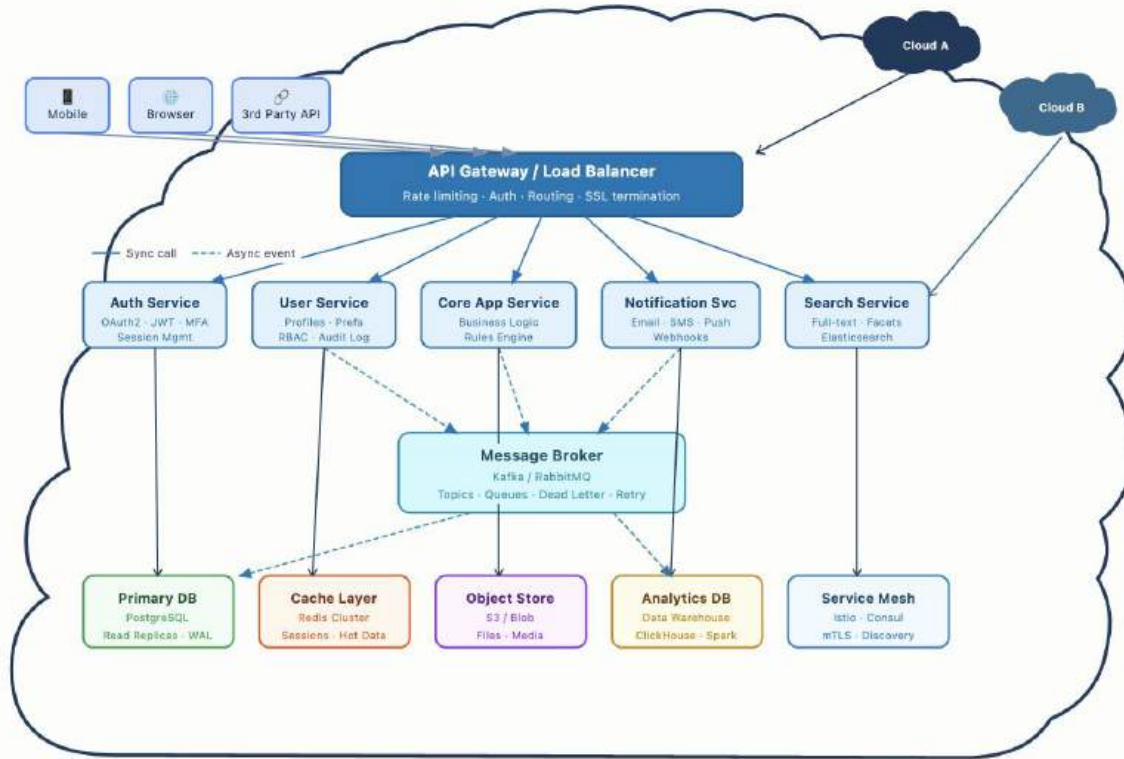


Definition: Observability is the practice of understanding a system's internal health and behavior by collecting and analyzing its external data outputs, specifically **logs, metrics, and traces**.

Purpose: It is crucial for decoding performance and tracking issues across complex, modern multi-cloud architectures.

Outcome: A robust observability solution enables teams to track performance across applications, services, and infrastructure, helping them quickly detect and respond to issues.

Applications are complex



Exemplary Architecture of a Complex Application

And you can't get the answers you need

What is my system doing?

Is it working or not?



SRE/DevOps

How do I fix it?



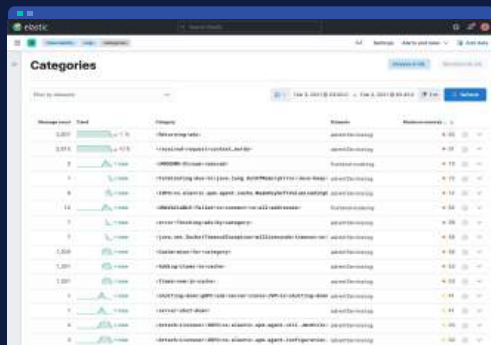
SRE/Dev

Who is impacted?

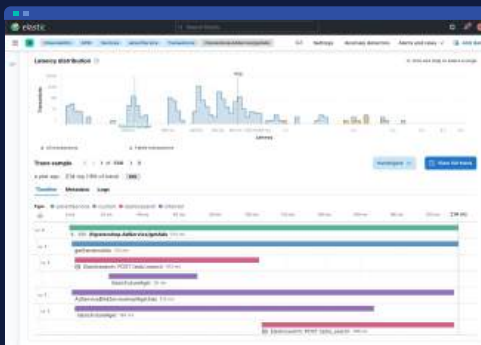


SRE/CIO

You need signals to get answers



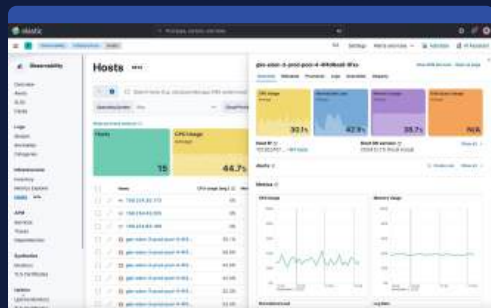
Logging



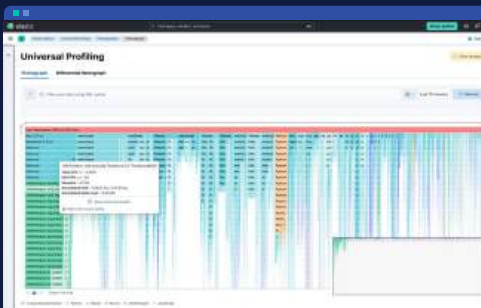
Tracing



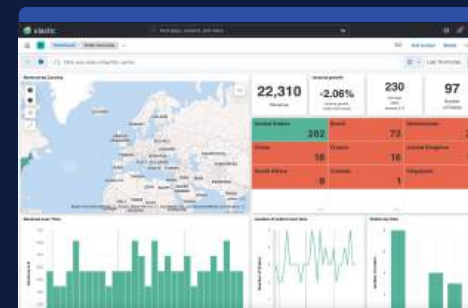
RUM



Infra metrics



Profiling



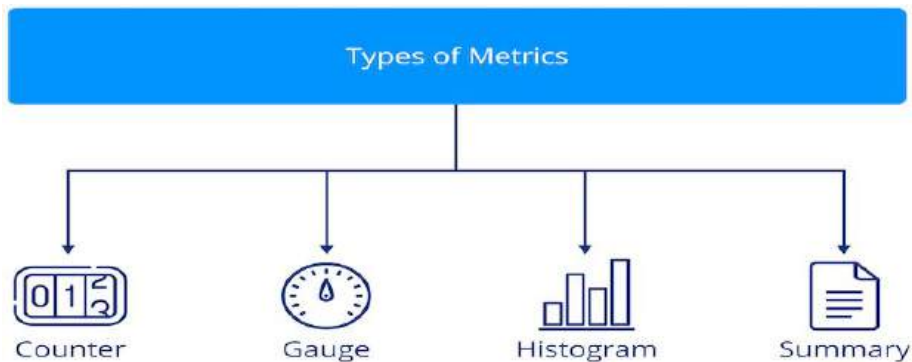
KPI metrics

Metrics

Metrics ("Is there a problem?")

What they are: Numeric data measured over intervals of time (e.g CPU utilization, memory usage, request, error rates).

Why they matter: They are lightweight, easy to store, and perfect for triggering real-time alerts and populating dashboards to show overall system health.



Logs

Logs ("What happened?")

What they are: Logs are structured and unstructured data from your infrastructure, applications, networks, and systems made up of timestamped entries relating to specific events.

Why they matter: When a metric spikes, logs provide the granular context. They tell you exactly what a specific service was doing at the exact moment things went wrong.

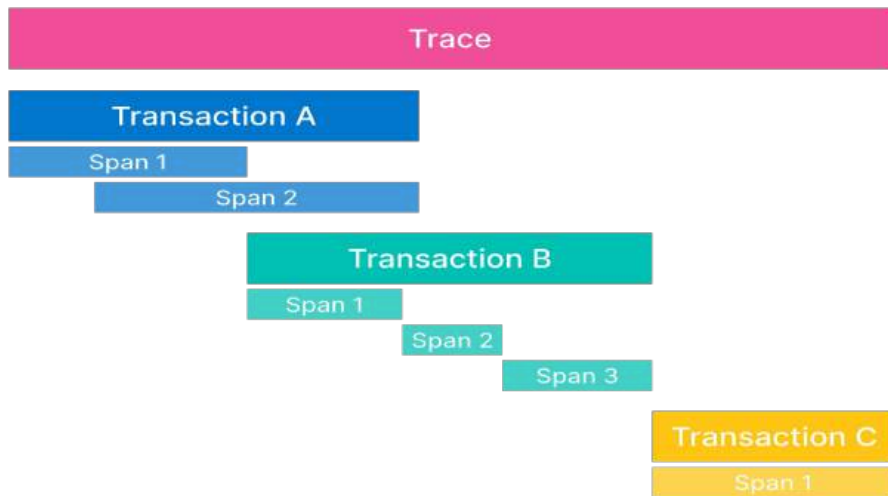
```
rabbit-1 | 2024-02-15 10:43:46.110217-00:00 [info] <0.553.0> Starting message stores for vhost '/'
rabbit-1 | 2024-02-15 10:43:46.110889-00:00 [info] <0.553.0> Message store '628807C1F09011660000/msg_store_transient': using rabbit_msg_store_ets_index to provide index
rabbit-1 | 2024-02-15 10:43:46.110887-00:00 [info] <0.553.0> Started message store of type transient for vhost '/'
rabbit-1 | 2024-02-15 10:43:46.110887-00:00 [info] <0.557.0> Message store '628807C1F09011660000/msg_store_persistent': using rabbit_msg_store_ets_index to provide index
rabbit-1 | 2024-02-15 10:43:46.111200-00:00 [warning] <0.167.0> Message store '628807C1F09011660000/msg_store_persistent': rebuilding indices from scratch
rabbit-1 | 2024-02-15 10:43:46.111315-00:00 [info] <0.553.0> Started message store of type persistent for vhost '/'
rabbit-1 | 2024-02-15 10:43:46.111504-00:00 [info] <0.553.0> Recovering 8 queues of type rabbit_classic_queue took 2ms
rabbit-1 | 2024-02-15 10:43:46.111583-00:00 [info] <0.553.0> Recovering 8 queues of type rabbit_queue_type took 0ms
rabbit-1 | 2024-02-15 10:43:46.111685-00:00 [info] <0.553.0> Recovering 8 queues of type rabbit_stream_queue took 0ms
rabbit-1 | 2024-02-15 10:43:46.112596-00:00 [info] <0.230.0> Created user 'qualizer'
rabbit-1 | 2024-02-15 10:43:46.113054-00:00 [info] <0.230.0> Successfully set user tags for user 'qualizer' to [administrator]
rabbit-1 | 2024-02-15 10:43:46.113680-00:00 [info] <0.230.0> Successfully set permissions for user 'qualizer' in virtual host '/' to '.*', '.*', '.*'
rabbit-1 | 2024-02-15 10:43:46.113651-00:00 [info] <0.230.0> Running boot step rabbit_observer_cli defined by app rabbit
rabbit-1 | 2024-02-15 10:43:46.113685-00:00 [info] <0.230.0> Running boot step rabbit_looking_glass defined by app rabbit
logger-migrator | 2024-02-15 10:43:43:02.702132002 level=info msg="Executing migration" id="Add index for created in annotation table"
logger-migrator | 2024-02-15 10:43:43:02.704060172 level=info msg="Executing migration" id="Add index for updated in annotation table"
logger-migrator | 2024-02-15 10:43:43:02.707459964 level=info msg="Executing migration" id="Convert existing annotations from seconds to milliseconds"
logger-migrator | 2024-02-15 10:43:43:02.7080105422 level=info msg="Executing migration" id="Add epoch_end column"
logger-migrator | 2024-02-15 10:43:43:02.713801252 level=info msg="Executing migration" id="Add index for epoch_end"
logger-migrator | 2024-02-15 10:43:43:02.717013882 level=info msg="Executing migration" id="Merge epoch_end and the same as epoch"
logger-migrator | 2024-02-15 10:43:43:02.719774824 level=info msg="Executing migration" id="Move region to single row"
logger-migrator | 2024-02-15 10:43:43:02.721536112 level=info msg="Executing migration" id="Remove index org_id_epoch from annotation table"
logger-migrator | 2024-02-15 10:43:43:02.723525762 level=info msg="Executing migration" id="Remove index org_id_dashboard_id_epoch from annotation table"
logger-migrator | 2024-02-15 10:43:43:02.725590272 level=info msg="Executing migration" id="Add index for org_id_dashboard_id_epoch_end_epoch on annotation table"
logger-migrator | 2024-02-15 10:43:43:02.729496664 level=info msg="Executing migration" id="Add index for org_id_epoch_end_epoch on annotation table"
logger-migrator | 2024-02-15 10:43:43:02.731612222 level=info msg="Executing migration" id="Remove index org_id_epoch_epoch_end from annotation table"
logger-migrator | 2024-02-15 10:43:43:02.734213082 level=info msg="Executing migration" id="Add index for alert_id on annotation table"
logger-migrator | 2024-02-15 10:43:43:02.736632422 level=info msg="Executing migration" id="Increase tags column to length 4096"
logger-migrator | 2024-02-15 10:43:43:02.741084862 level=info msg="Executing migration" id="create test data table"
logger-migrator | 2024-02-15 10:43:43:02.753715652 level=info msg="Executing migration" id="create dashboard version table v1"
```


Traces

Traces ("Where did it happen?")

What they are: The end-to-end journey of a single request as it moves through a distributed system (like a microservices architecture).

Why they matter: A single user click might trigger calls across dozens of downstream services. Distributed tracing allows DevOps and SRE teams to map that entire path, revealing exactly which service caused a bottleneck or threw an error.



The Three Pillars

Observability



Observability Use Case



Log monitoring & analytics



Cloud & infrastructure
monitoring



Application Performance
Monitoring



Digital Experience Monitoring



Universal Profiling

Observability Use Case



AI-powered log analysis with Streams



Anomaly Detection & Alerting



Incident response and management



LLM Observability



AIOps & AI Agent

Application Performance Monitoring

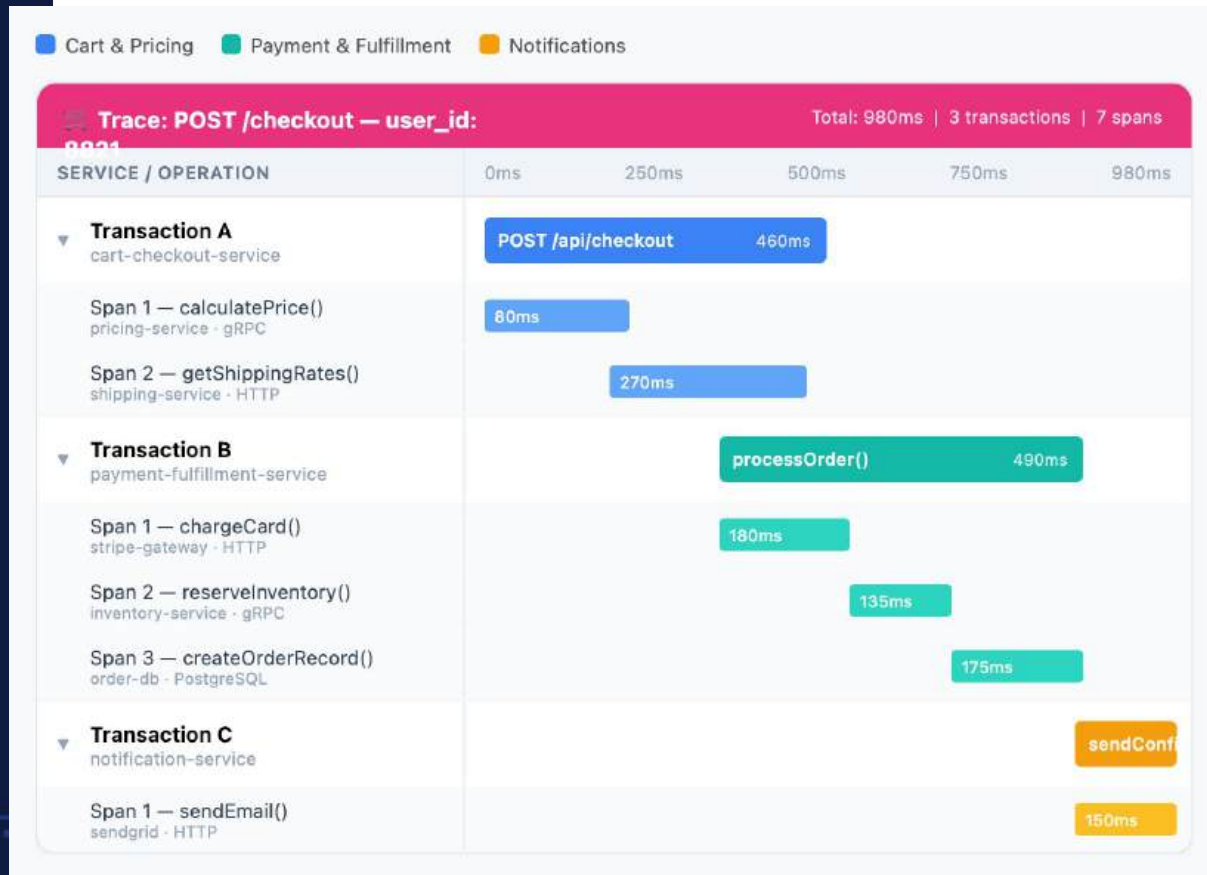
What is APM?

- **Built on the Elastic Stack:** Elastic APM is an application performance monitoring system integrated directly with the Elastic Stack.
- **Real-Time Monitoring:** It tracks software services and applications in real time to provide up-to-the-minute insights.
- **Detailed Data Collection:** It captures specific performance metrics, including response times for incoming requests, database queries, cache calls, and external HTTP requests.
- **Efficient Troubleshooting:** By gathering this granular data, it allows teams to quickly pinpoint and resolve performance bottlenecks.

Data Types

- **Transaction:** A transaction describes an event captured by an Elastic APM agent instrumenting a service.
- **Span:** A span contains information about the execution of a specific code path like DB calls, HTTP requests
- **Traces:** A trace is a group of *transactions* and *spans*
- **Errors:** This event contains information to help determine where, why and what error occurred
- **Metrics:** Metrics measure the state of a system by gathering information on a regular interval.

Data Flow





Data Collection



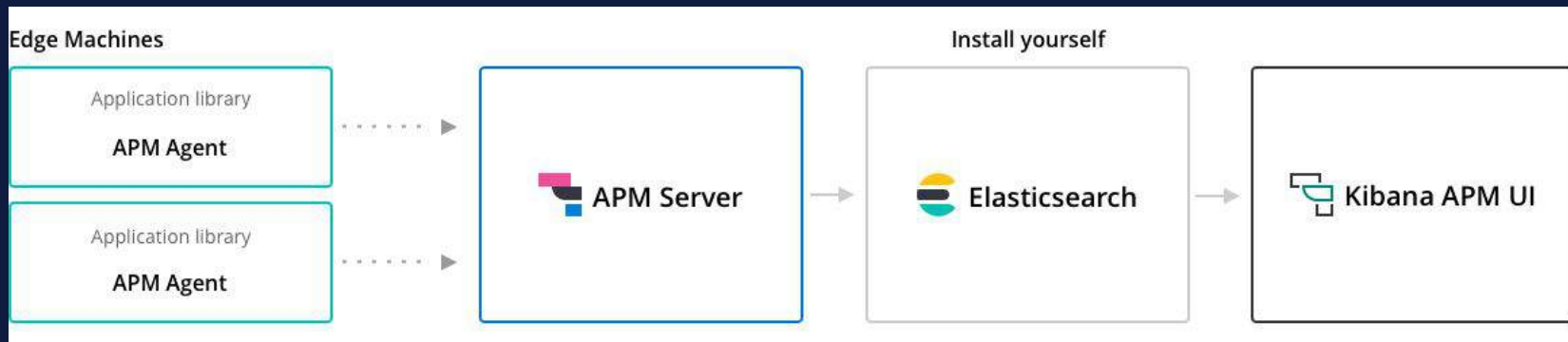
Elastic APM Agents

Elastic APM agents are instrumentation libraries written in the same language as your service.

OpenTelemetry

OpenTelemetry is an open source set of APIs, SDKs, tooling, and integrations that enable the capture and management of telemetry data from your services and applications.

Elastic APM Overview

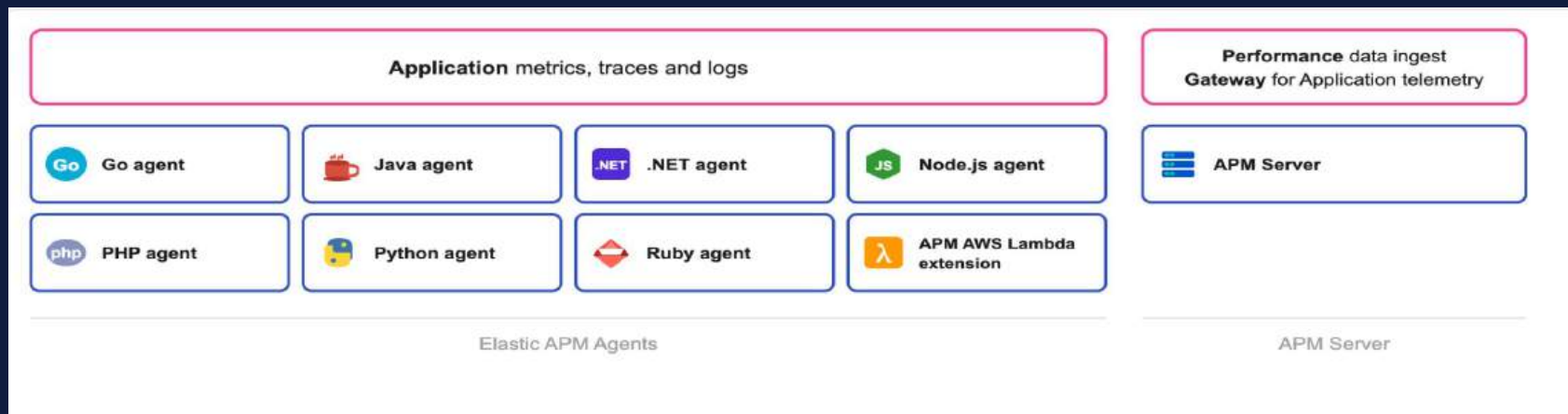


- APM Agents for several programming languages with Auto instrumentation (*)
- APM Server standalone or APM Integration under Elastic Agent

Collects performance data from application in real-time

- Response times
- Database queries
- External HTTP requests
- Unhandled errors and exceptions

Elastic APM Components



OpenTelemetry & EDOT

What is OpenTelemetry (OTel)?

- **OpenTelemetry**, also known as OTel, is a vendor-neutral open source [Observability](#) framework
- **OpenTelemetry** used for generating, collecting, and exporting telemetry data such as [traces](#), [metrics](#), and [logs](#).
- **OpenTelemetry** is a CNCF community-driven open-source project (Cloud Native Computing Foundation, the folks in charge of Kubernetes).
- **OpenTelemetry** is actually successful in reducing the number of competing standards

What is OpenTelemetry (OTel)?

- [Open Telemetry Contributions](#)

OpenTelemetry Companies statistics (Contributions, Range: Last year), bots excluded ▾		
Rank ^	Company	Number
	All	115615
1	Splunk Inc.	19741
2	Elasticsearch Inc.	15326
3	Microsoft Corporation	13762
4	Grafana Labs	11730
5	Datadog Inc	5981



Why OpenTelemetry (OTel)?

- **Unified telemetry:** Combines tracing, logging, and metrics into a single framework enabling correlation of all data and establishing an open standard for telemetry data.
- **Vendor-neutrality:** Integration with different backends for processing the data.
- **Cross-platform:** Supports various languages (Java, Python, Go, etc.) and platforms, making it versatile for different development environments.



OTel Specification

- **API**

- defines the interfaces and constants outlined in the specification
- used by application and library developers for vendor-agnostic instrumentation
- refers to a no-op implementation by default

- **SDK**

- provider implements the OpenTelemetry API
- contains the actual logic to generate, process and emit telemetry
- OpenTelemetry ships with official providers that serve as the reference implementation (commonly referred to as the SDK)
- it is possible to write your own

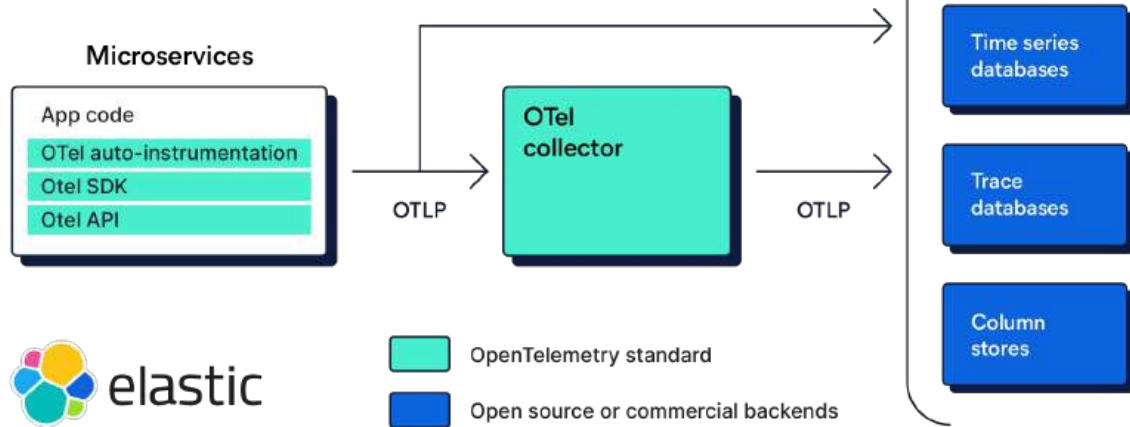
OTel Specification

Wire Protocol:

- The OpenTelemetry Protocol (OTLP) is an open source and vendor-neutral wire format that defines:
 - how data is encoded in memory
 - a protocol to transport that data across the network
- OTLP offers three transport mechanisms for transmitting telemetry data: HTTP/1.1, HTTP/2, and gRPC.
- OTLP data is often encoded using the Protocol Buffers (Protobuf) binary format, which is compact and efficient for network transmission

OTel Architecture

Example of OpenTelemetry implementation and how to receive, process and export telemetry data



Elastic Distributions of OpenTelemetry (EDOT)

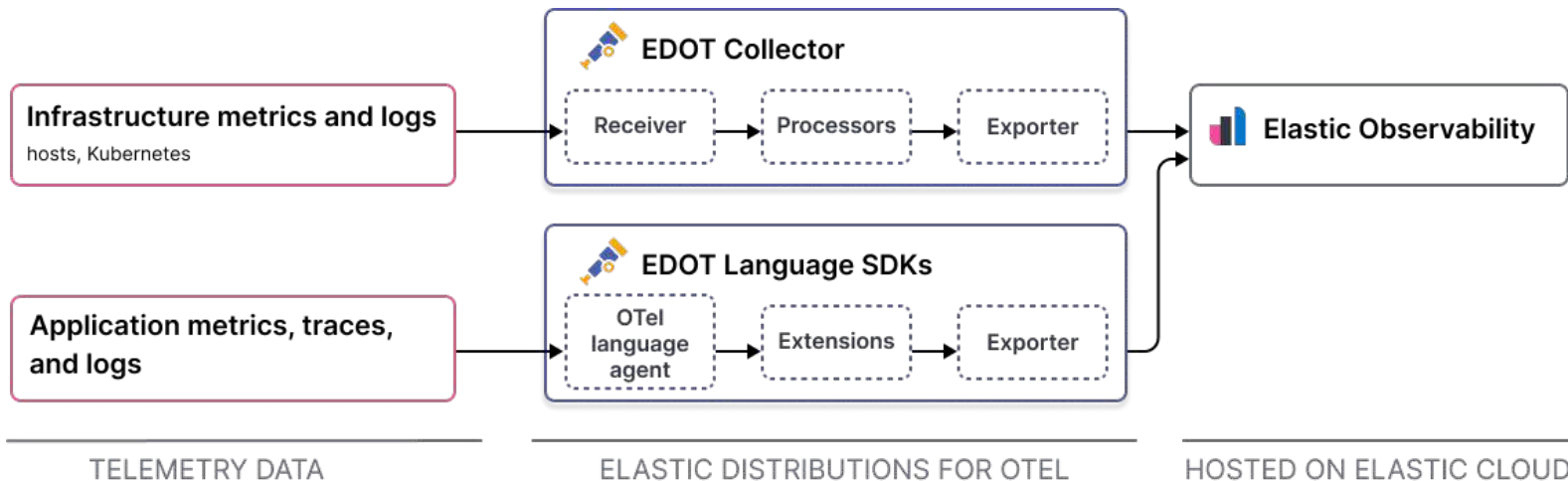
Elastic distribution of OpenTelemetry Collector

Essential components for seamless OTEL connectivity to Elastic
Out of the box logging, host metrics and K8s enrichment configurations

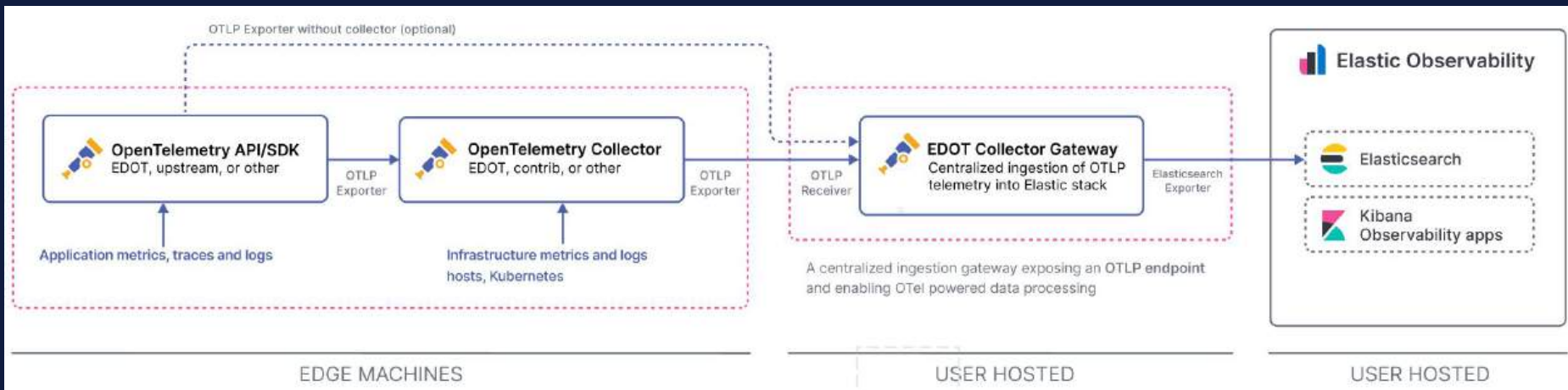
Elastic distribution of OpenTelemetry APM agents

**EDOT - OTEL with
enterprise support**

[EDOT github](#)

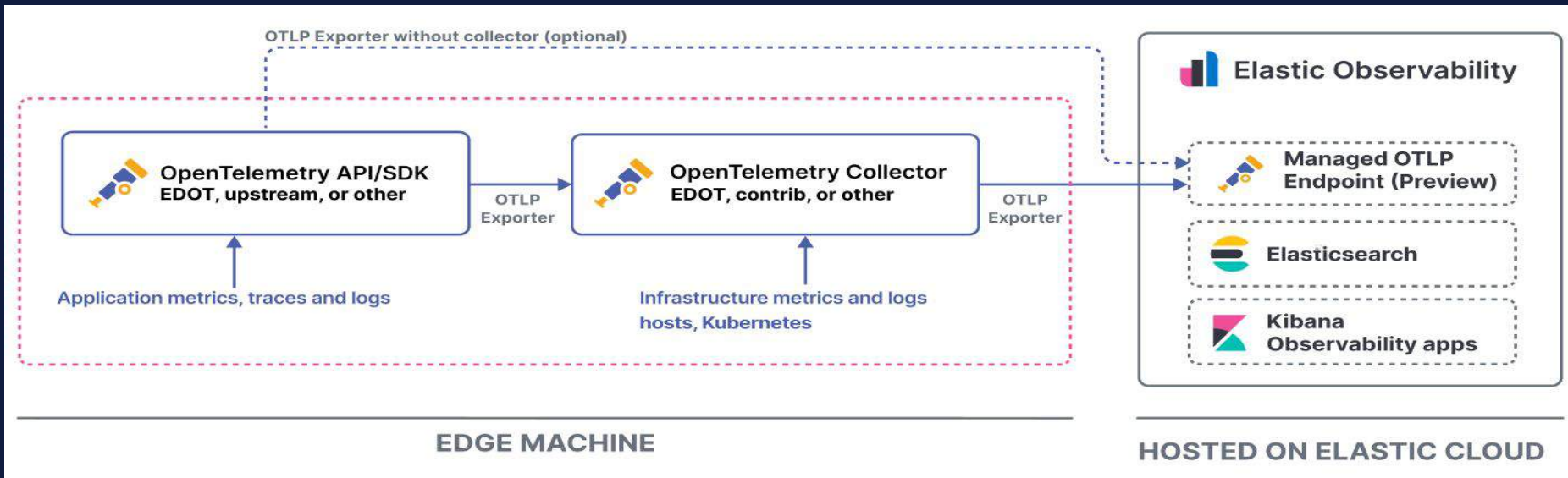


Data Collection (EDOT) - Self Managed



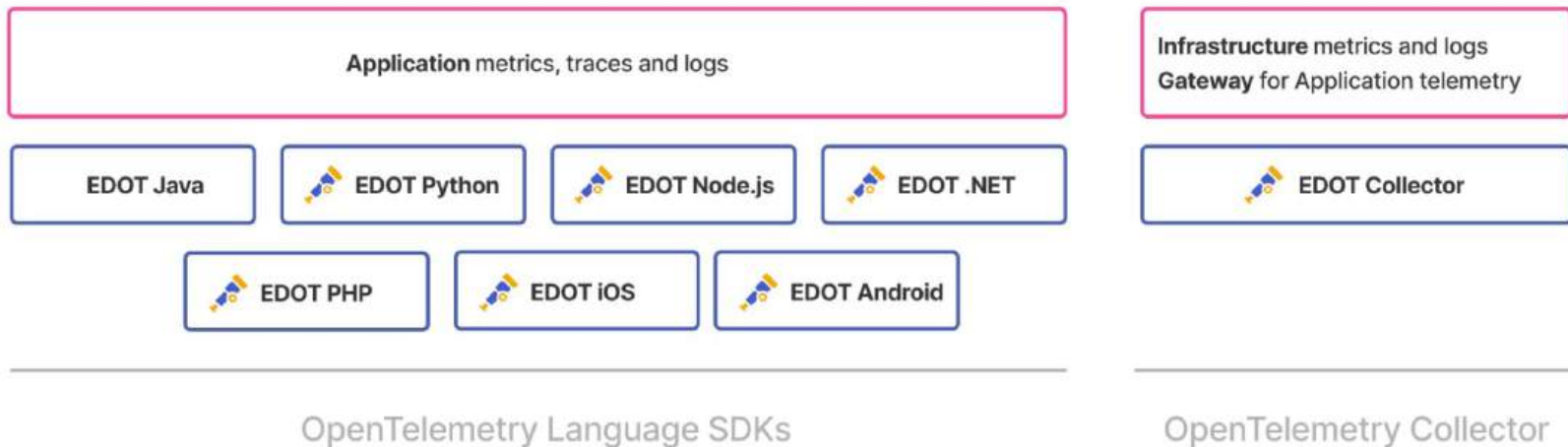
- Self Hosted , ECE or ECK can have EDOT Collector as Gateway with inbuilt handling of data to export to Elasticsearch
- Up stream can be either EDOT or Standard OTEL component SDK or Collector

Data Collection (EDOT) - ECH/Serverless



- ECH and Elastic Server has Managed OTEL Collector which accepts OTLP standard and scale as per need for Ingestion
- Up stream can be either EDOT or Standard OTEL component SDK or Collector

Data Collection (EDOT Agent)



Visualize APM Data

Services

Services

[Tell us what you think!](#)

All services Service groups

Environment All

Inventory Service Map

Search transactions, errors and metrics (E.g. transaction.duration.us > 300000 AND

Last 15 minutes

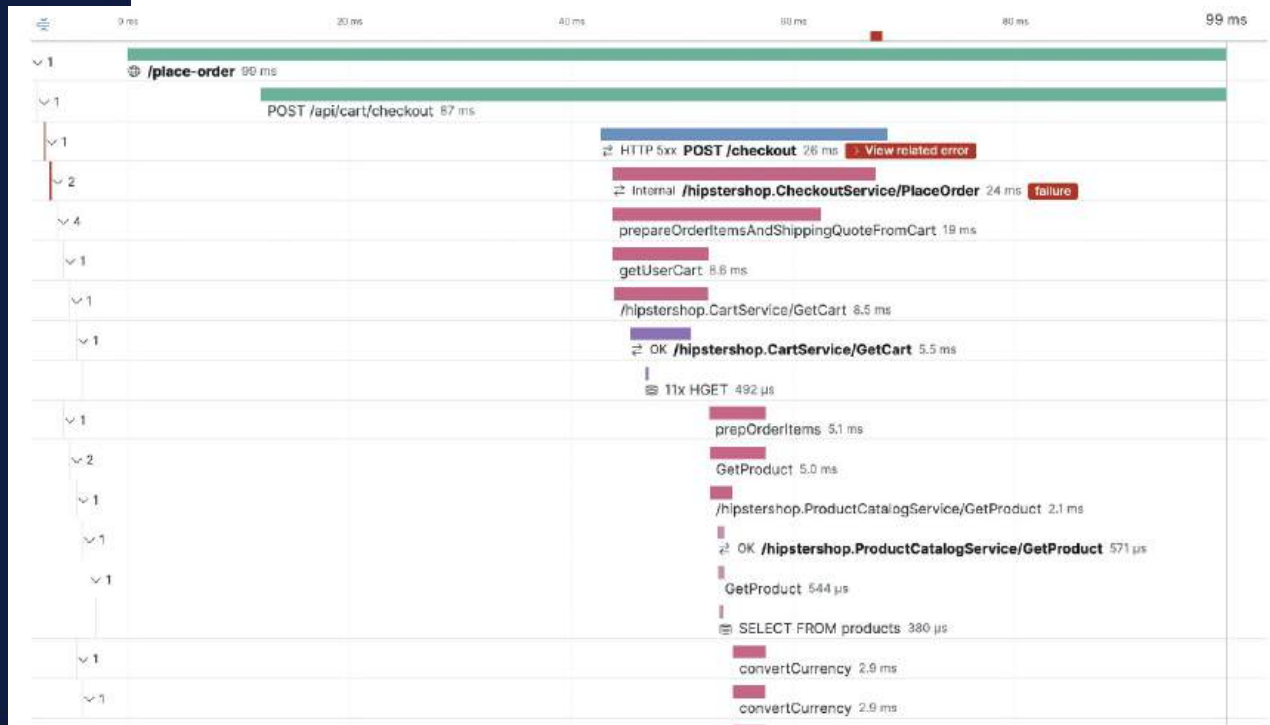
☒ Comparison Day before

[What are these metrics?](#)

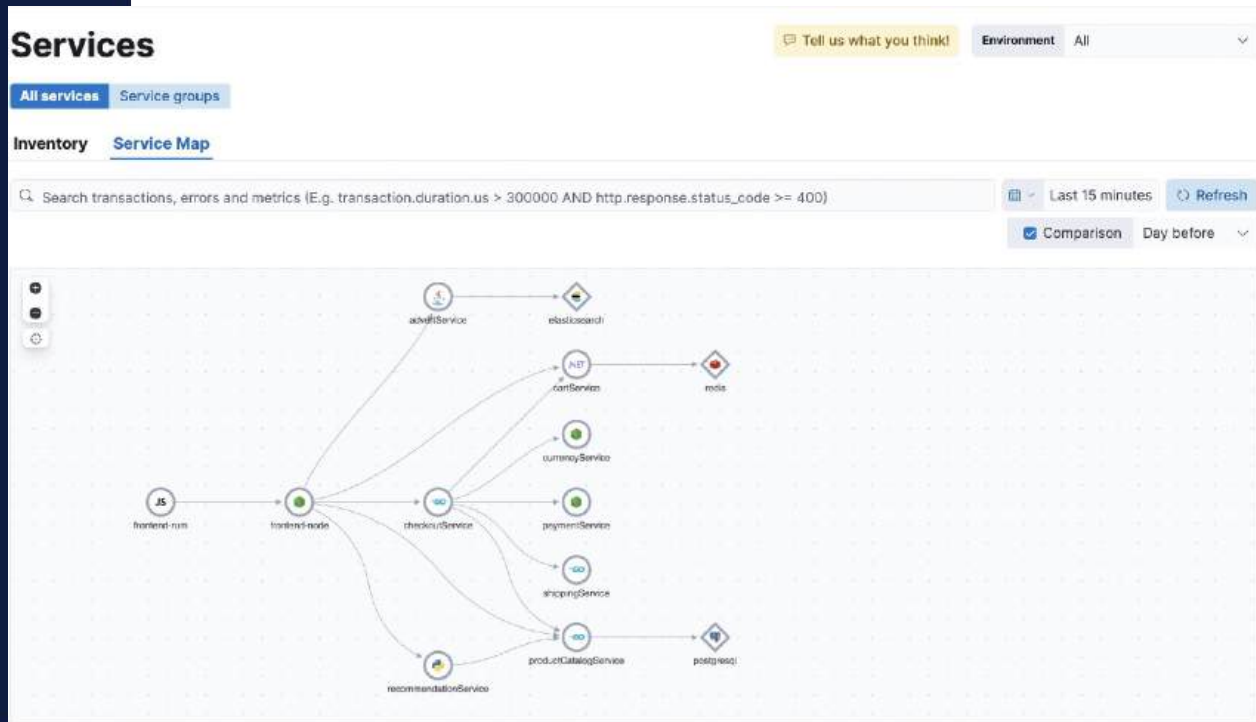
Search services by name

Name	Environment	Latency (avg.)	Throughput	Failed transaction rate
 frontend-node	ecommerce	183 ms	116.7 tpm	5.9%
 productCatalogSer...	ecommerce	217 ms	95.1 tpm	0%
 JS frontend-rum	ecommerce	1,989 ms	27.5 tpm	N/A
 NET cartService	ecommerce	5.3 ms	20.3 tpm	0%
 recommendationSe...	ecommerce	1,005 ms	13.7 tpm	0%

Traces



Service Map



Dependencies

Dependencies

Tell us what you think!

Environment All

Search dependency metrics (e.g. span.destination.service.resource:elasticsearch)

Last 15 minutes

Refresh

Comparison

Day before

Dependency	Latency (avg.)	Throughput	Failed transaction rate	Impact
 elasticsearch	28 ms 	6.7 tpm 	100% 	
 postgresql	1.0 ms 	115.7 tpm 	0% 	
 redis	0.5 ms 	160.1 tpm 	0% 	

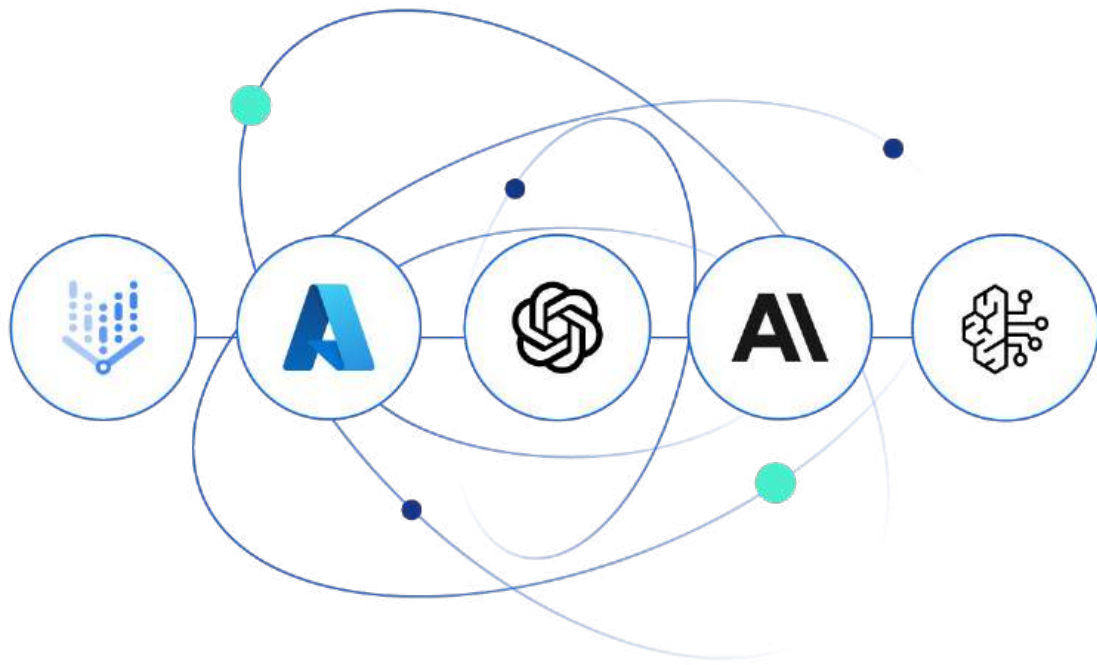
Rows per page: 25

< 1 >

LLM & Agentic AI Observability

LLM Observability

Observe your agentic AI
stack





LLM Observability



- **LLM observability** is the practice of monitoring, tracing, and evaluating large language models in real-time
- While transformative, LLMs introduce unique challenges in **reliability, performance, and cost management**.
- Traditional monitoring tools are no longer enough; they must evolve into advanced **observability capabilities** to keep models efficient.



LLM Observability



*Microsoft CEO Satya Nadella has actively warned against "**tokenmaxxing**"—the uncritical and costly consumption of AI tokens by running frontier models for routine tasks. While admitting he finds the habit "addictive," he urges employees and businesses to match task complexity to the right model and focus on business value rather than vanity token counts*



LLM Observability



Real-time monitoring and tracing

- Captures traces, spans, and agent workflows to provide visibility into otherwise opaque LLM operations.
- Records critical metadata including inputs, outputs, latency, errors, and privacy signals.
- Tracks step-by-step logic across model calls, tool interactions, and database retrievals.
- Collects and unifies system logs, metrics, and traces to continuously evaluate model health.



LLM Observability

Performance Metrics

When evaluating LLM performance, critical metrics include latency, throughput, token usage, error rates, and overall system efficiency.

- **Latency:** Identifies the time spent between input and output, and potential bottlenecks.
- **Throughput:** Identifies how many requests a model processes within a given time period.
- **Token usage:** Monitors how many tokens were used in processing a request.
- **Error rates:** Measures how reliable a model is based on the rate of failed responses.



LLM Observability



Cost Management

- SREs must closely manage LLM costs to ensure configurations remain cost-effective.
- **Total Invocations:** Monitors request volume to identify spike patterns or inefficient application loops.
- **Token Usage:** Tracks total token consumption to calculate and forecast operational spend
- **Granular Time-Series Breakdowns:** Segregates usage data by specific models and API endpoints, allowing teams to instantly pinpoint which exact configurations are driving up costs.



LLM Observability

Security & Compliance

Prompt Injections: Surfaces dashboard data on malicious prompts designed to hijack model behavior.

PII Leakage: Records and displays incidents where sensitive credentials or personal data were intercepted.

Compliance:

- User interactions prompt policy violations
- Blocked potentially harmful responses



LLM Observability



- **Comprehensive Framework:** Elastic provides a robust framework utilizing **metrics, logs, and traces** to keep applications reliable, efficient, and easy to troubleshoot.
- **Instant Insights:** Features pre-configured, **out-of-the-box dashboards** that deliver deep visibility into:
 - Model prompts and responses
 - Performance and system usage
 - Operational costs



LLM Observability



Elastic achieves end-to-end LLM observability through two primary methods:

- **Integrations:** Capturing metrics and logs for LLM APIs via built-in Elastic integrations to track performance, usage, and costs.
- **Instrumentation:** APM tracing directly to LLM models to capture deep insights

LLM Observability - Elastic Integrations

AI platform observability with Elastic integrations



Google Vertex AI



Open AI



Azure OpenAI



Amazon Bedrock



Anthropic



LLM Observability - Tracing



- End-to-End Tracing & Debugging for AI Apps
- Monitor, analyze, and debug artificial intelligence applications using Elastic APM paired with OpenTelemetry (OTel) for
 - Python
 - Java
 - and Node.js
- Out-of-the-box compatibility with specialized LLM frameworks like LangTrace, OpenLIT, and OpenLLMetry.



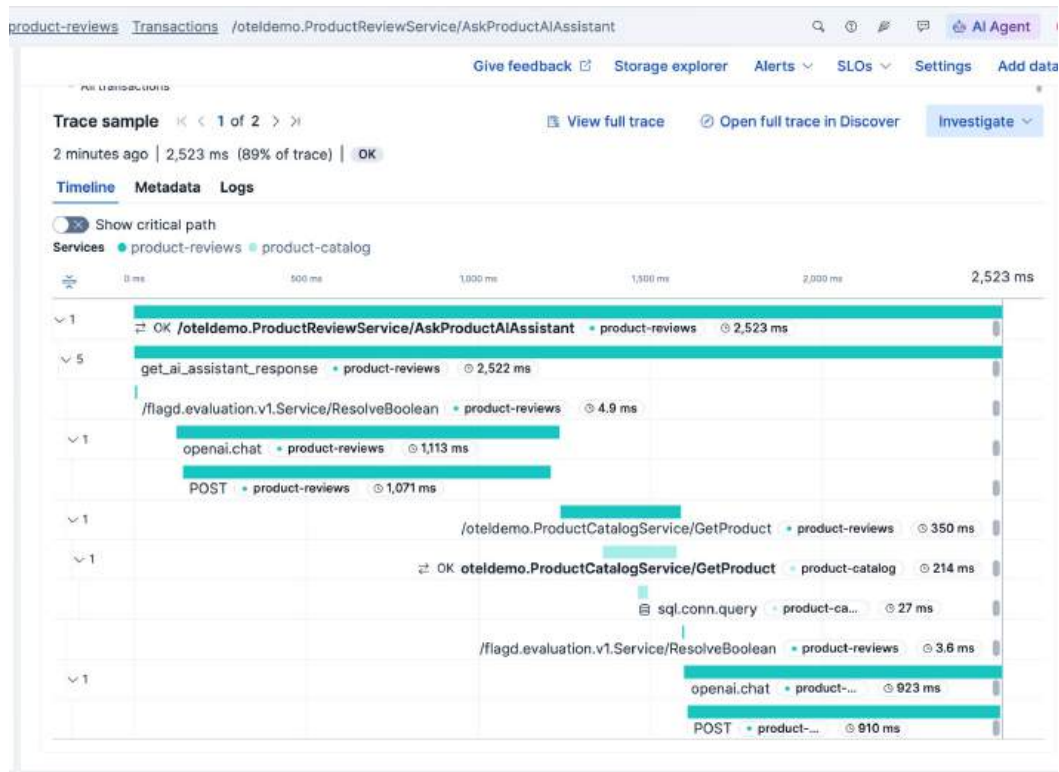
LLM Observability - Tracing



Transaction and span of a request, tracing shows critical information such as,

- Specific models utilized
- Request duration
- Errors encountered
- Tokens used per request
- Prompts and responses between the LLM

LLM Observability - Tracing



Demo LLM Observability & AI Assistant